

Proiect - Inteligență Artificială

Algoritmul de verificare înainte (Forward Checking) aplicat în jocul Futoshiki

Studenti: Rotaru Ionut-Rares, Stoian Mario-Daniel

Grupa: 1406A

Anul: IV

Cuprins

Descrierea problemei.....	3
Aspecte teoretice privind algoritmul	3
Modalitatea de rezolvare.....	4
Listarea părților semnificative din codul.....	4
Rezultate	7
Concluzii	9
Contributii	10
Bibliografie	10

Descrierea problemei

Problema abordată în acest proiect este rezolvarea și asistarea utilizatorului în jocul de logică Futoshiki. Acesta este un puzzle japonez jucat pe o grilă de dimensiune $N \times N$, având următoarele reguli:

1. **Unicitate:** Fiecare cifră de la 1 la N trebuie să apară o singură dată pe fiecare linie și pe fiecare coloană (structură de Pătrat Latin)
2. **Inegalități:** Între anumite celule adiacente există semne de inegalitate ($<$ sau $>$). Cifrele completate trebuie să respecte aceste relații.

Din perspectiva Inteligenței Artificiale, Futoshiki este o problemă de satisfacere a constrângerilor (Constraint Satisfaction Problem - CSP). Obiectivul este de a găsi o atribuire completă a variabilelor (celulelor) care să satisfacă simultan toate constrângerile binare (de inegalitate) și globale (de unicitate).

Proiectul își propune să implementeze o aplicație interactivă care nu doar validează regulile, ci folosește algoritmi specifici CSP pentru a calcula în timp real opțiunile valide rămase pentru fiecare căsuță necompletată.

Aspecte teoretice privind algoritmul

Algoritmul central utilizat este Forward Checking (Verificare Înainte). Acesta este o optimizare a algoritmului clasic de Backtracking, având rolul de a reduce spațiul de căutare prin detectarea timpurie a eșecurilor.

Principiul de funcționare:

În momentul în care unei variabile X (o celulă curentă) i se atribuie o valoare v :

1. Algoritmul identifică toate variabilele viitoare Y (celule necompletate) care sunt conectate de X printr-o constrângere (sunt pe aceeași linie/coloană sau au o relație de inegalitate).
2. Din domeniul variabilelor Y sunt eliminate temporar toate valorile care intră în conflict cu atribuirea $X = v$.
3. Dacă domeniul oricărei variabile Y devine vid (nu mai are nicio valoare posibilă), atribuirea curentă $X = v$ este considerată eronată, iar algoritmul împiedică această mutare sau face pasul înapoi (backtrack).

Avantaje față de verificarea simplă:

Spre deosebire de o verificare care verifică doar dacă mutarea curentă e validă, Forward Checking "privește în viitor". Acest lucru permite utilizatorului să observe efectul de propagare: plasarea unei cifre într-un colț poate reduce opțiunile disponibile într-o zonă opusă a tablei.

Modalitatea de rezolvare

Aplicația a fost dezvoltată în limbajul C# folosind framework-ul Windows Forms pentru interfața grafică. Soluția este structurată pe două niveluri:

1. Backend (Logic):
 - a. Gestionarea stării jocului printr-o matrice de celule (CellGrid).
 - b. Generarea procedurală a nivelurilor: Se creează mai întâi o soluție validă completă în memorie, apoi se deduc semnele de inegalitate pe baza acestora și se ascund cifrele. Aceasta garantează că puzzle-ul generat are cel puțin o soluție.
 - c. Implementarea algoritmului Forward Checking în metoda GetPosibleValues, care filtrează dinamic domeniile.
2. Frontend (Interfață):
 - a. Utilizarea unor controale personalizate (FutoshikiCell) care afișează atât valoarea curentă (mare), cât și domeniul de valori posibile (mic, în partea de jos).
 - b. Sistem de input bazat pe tastatură, cu validare instantanee.
 - c. Feedback vizual: Semnele de inegalitate sunt desenate dinamic între celule.

Listarea părților semnificative din codul

Mai jos sunt prezentate secvențele de cod esențiale pentru funcționarea algoritmului.

A. Implementarea Forward Checking (Logica de filtrare)

```
// Funcție ce implementează Forward Checking pentru a returna o lista de valori posibile
2 references
public List<int> GetPosibleValues(CellGrid grid, int size)
{
    List<int> domeniu = Enumerable.Range(1, size).ToList();

    //Verificam linia pe care se afla celula si eliminam valorile existente din domeniu
    for(int c = 0; c < size; c++)
    {
        if(c != this.Col && grid.GetCell(this.Row, c).Value != 0)
        {
            domeniu.Remove(grid.GetCell(this.Row, c).Value);
        }
    }

    //Verificam si coloana pe care se afla celula
    for(int r = 0; r < size; r++)
    {
        if(r != this.Row && grid.GetCell(r, this.Col).Value != 0)
        {
            domeniu.Remove(grid.GetCell(r, this.Col).Value);
        }
    }

    //Verificam lista de constrangeri
    foreach(var constr in grid.GetConstraints())
    {
        // Verificam daca celula are vreo constrangeri
        bool isCell = (constr.Row1 == this.Row && constr.Col1 == this.Col);
        bool isCell2 = (constr.Row2 == this.Row && constr.Col2 == this.Col);
        if(!isCell && !isCell2)
        {
            continue;
        }

        // Daca are, determinam vecinul si relatia celulei fata de el
        FutoshikiCell vecin = null;
        char sign = ' ';
        if(isCell)
        {
            vecin = grid.GetCell(constr.Row2, constr.Col2);
            switch (constr.Sign)
            {
                case '<': sign = '<'; break;
                case '>': sign = '>'; break;
                case 'v': sign = 'v'; break;
                case '^': sign = '^'; break;
            }
        }
        else if(isCell2)
        {
            vecin = grid.GetCell(constr.Row1, constr.Col1);
            switch (constr.Sign)
            {
                case '<': sign = '>'; break;
                case '>': sign = '<'; break;
                case 'v': sign = 'v'; break;
                case '^': sign = '^'; break;
            }
        }

        // Daca vecinul are valoare, eliminam din domeniu valorile care nu respecta constrangerea
        if (vecin != null)
        {
            if (vecin.Value != 0)
            {
                if (sign == '<')
                {
                    domeniu.RemoveAll(x => x >= vecin.Value);
                }
                else if (sign == '>')
                {
                    domeniu.RemoveAll(x => x <= vecin.Value);
                }
            }

            // Daca vecinul nu are valoare, facem o euristica si eliminam extremitatile
            else
            {
                if (sign == '<')
                {
                    domeniu.Remove(size);
                }
                else if (sign == '>')
                {
                    domeniu.Remove(1);
                }
            }
        }
    }

    // Returnam lista de posibilitati
    return domeniu;
}
```

Secvență de cod din clasa FutoshikiCell care implementează logica centrală a algoritmului Forward Checking. Metoda GetPosibleValues calculează lista de valori permise pentru o celulă prin trei pași:

1. Eliminarea valorilor deja existente pe linia și coloana curentă (proprietatea de Pătrat Latin).
2. Identificarea relațiilor de inegalitate ($<$, $>$, $\vee$, $\wedge$) cu celulele vecine.
3. Retezarea domeniului: eliminarea valorilor care nu respectă inegalitatea matematică față de vecinii care au deja o valoare atribuită.

B. Propagarea Domeniilor (Update Vizual)

După fiecare mutare validă, aplicația recalculează opțiunile pentru toată tabla

```

0 references
private void RefreshAllDomains()
{
    int size = (int)numSize.Value;
    for (int r = 0; r < size; r++)
    {
        for (int c = 0; c < size; c++)
        {
            grid.GetCell(r, c).GetDomain(grid, size);
        }
    }
}
1 reference

```

Metoda responsabilă de actualizarea stării jocului. Această funcție iterează prin toate celulele matricei ($N \times N$) și apelează pentru fiecare metoda GetDomain. Rolul ei este să asigure că, după orice mutare a utilizatorului (adăugare sau ștergere de cifră), algoritmul de Forward Checking este reaplicat global, iar utilizatorul vede instantaneu noile opțiuni valide rămase pentru fiecare căsuță necompletată.

C. Validarea Mutării și Detecția Blocajelor

Interceptarea tastelor utilizatorului și verificarea validității înainte de aplicare.

```

0 references
protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    if (selectedCell == null) return base.ProcessCmdKey(ref msg, keyData);

    if (keyData == Keys.Back)
    {
        if (!selectedCell.IsFixed)
        {
            selectedCell.SetValue(0);
            RefreshAllDomains();
        }
        return true;
    }

    int val = 0;
    if (keyData >= Keys.D1 && keyData <= Keys.D9)
        val = (int)keyData - (int)Keys.D0;
    else if (keyData >= Keys.NumPad1 && keyData <= Keys.NumPad9)
        val = (int)keyData - (int)Keys.NumPad0;

    if (val > 0)
    {
        int size = (int)numSize.Value;
        if (val > size) return true;
        List<int> validOptions = selectedCell.GetPossibleValues(grid, size);
        if (validOptions.Count == 0)
        {
            MessageBox.Show("Mutare imposibila pentru numerele selectate momentan nu se potriveste nimic aici.\nIncearca alte combinatii (selecteaza alte casute si sterge cifrele");
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning
        );
        }
        return true;
    }
}

```

```

    if (validOptions.Contains(val))
    {
        selectedCell.SetValue(val);
        RefreshAllDomains();
        CheckVictory(size);
    }
    else
    {
        MessageBox.Show($"Mutare invalida! Conform regulilor, aici poti pune doar: {string.Join(", ", validOptions)}");
    }
    return true;
}
return base.ProcessCmdKey(ref msg, keyData);
}

```

Mecanismul de interacțiune și validare a constrângerilor. Această metodă suprascrie funcția standard de procesare a tastelor pentru a implementa logica specifică jocului:

1. **Ștergerea (Backspace):** Permite utilizatorului să corecteze o mutare anterioară (dacă celula nu este fixă), declanșând recalcularea domeniilor.
2. **Validarea Input-ului:** Când utilizatorul apasă o cifră, algoritmul verifică mai întâi dacă mutarea ar duce la un blocaj (domeniu vid). Dacă există opțiuni, verifică dacă cifra aleasă este permisă de regulile curente (calculate prin GetPossibleValues).
3. **Feedback:** Dacă mutarea este validă, se aplică și se propagă efectul. Dacă nu, utilizatorul primește un mesaj de eroare explicativ, prevenind stările invalide.

D. Generarea Procedurală a Nivelului

```

1 reference
private void GenerateBoardUI(int size)
{
    pnlBoard.Controls.Clear();
    this.Width = 200 + size * 105;
    this.Height = 100 + size * 105;
    this.CenterToScreen();

    grid = new CellGrid(size);
    hSigns = new Label[size, size - 1];
    vSigns = new Label[size - 1, size];

    int cellSize = 60;
    int gap = 45;
    int startX = 20;
    int startY = 20;

    int[,] solution = new int[size, size];
    for (int r = 0; r < size; r++)
    {
        for (int c = 0; c < size; c++)
        {
            solution[r, c] = ((r + c) % size) + 1;
        }
    }

    for (int r = 0; r < size; r++)
    {
        for (int c = 0; c < size; c++)
        {
            FutoshikiCell cell = new FutoshikiCell();
            cell.Row = r;
            cell.Col = c;
            int x = startX + c * (cellSize + gap);
            int y = startY + r * (cellSize + gap);
            cell.Location = new Point(x, y);
            cell.Clicked += OnCellClicked;
            pnlBoard.Controls.Add(cell);
            grid.SetCell(cell);

            if (rng.Next(100) < 20)
            {
                cell.SetValue(solution[r, c]);
                cell.IsFixed = true;
            }
        }

        if (c < size - 1)
        {
            Label lblH = CreateSignLabel();
            lblH.Location = new Point(x + cellSize + (gap / 2) - 15, y + (cellSize / 2) - 20);
            pnlBoard.Controls.Add(lblH);
            hSigns[r, c] = lblH;
        }

        if (r < size - 1)
        {
            Label lblV = CreateSignLabel();
            lblV.Location = new Point(x + (cellSize / 2) - 12, y + cellSize + (gap / 2) - 20);
            pnlBoard.Controls.Add(lblV);
            vSigns[r, c] = lblV;
        }
    }

    for (int r = 0; r < size; r++)
    {
        for (int c = 0; c < size; c++)
        {
            int x = startX + c * (cellSize + gap);
            int y = startY + r * (cellSize + gap);
            if (c < size - 1)
            {
                if (rng.Next(100) < difficulty)
                {
                    Label lblH = CreateSignLabel();
                    lblH.Location = new Point(x + cellSize + (gap / 2) - 15, y + (cellSize / 2) - 20);
                    pnlBoard.Controls.Add(lblH);
                    hSigns[r, c] = lblH;
                    char semn;
                    if (solution[r, c] < solution[r, c + 1]) semn = '<';
                    else semn = '>';

                    lblH.Text = semn.ToString();
                    grid.AddConstraint(grid.GetCell(r, c), grid.GetCell(r, c + 1), semn);
                }
            }

            if (r < size - 1)
            {
                if (rng.Next(100) < difficulty)

```

```
    }
    for (int r = 0; r < size; r++)
    {
        for (int c = 0; c < size; c++)
        {
            int x = startX + c * (cellSize + gap);
            int y = startY + r * (cellSize + gap);
            if (c < size - 1)
            {
                if (rng.Next(100) < difficulty)
                {
                    Label lblH = CreateSignLabel();
                    lblH.Location = new Point(x + cellSize + (gap / 2) - 15, y + (cellSize / 2) - 20);
                    pnlBoard.Controls.Add(lblH);
                    hSigns[r, c] = lblH;
                    char semn;
                    if (solution[r, c] < solution[r, c + 1]) semn = '<';
                    else semn = '>';

                    lblH.Text = semn.ToString();
                    grid.AddConstraint(grid.GetCell(r, c), grid.GetCell(r, c + 1), semn);
                }
            }
            if (r < size - 1)
            {
                if (rng.Next(100) < difficulty)
                {
                    Label lblV = CreateSignLabel();
                    lblV.Location = new Point(x + (cellSize / 2) - 12, y + cellSize + (gap / 2) - 20);
                    pnlBoard.Controls.Add(lblV);
                    vSigns[r, c] = lblV;
                    char semn;
                    if (solution[r, c] < solution[r + 1, c]) semn = '^';
                    else semn = 'v';

                    lblV.Text = semn.ToString();
                    grid.AddConstraint(grid.GetCell(r, c), grid.GetCell(r + 1, c), semn);
                }
            }
        }
    }
    RefreshAllDomains();
}
```

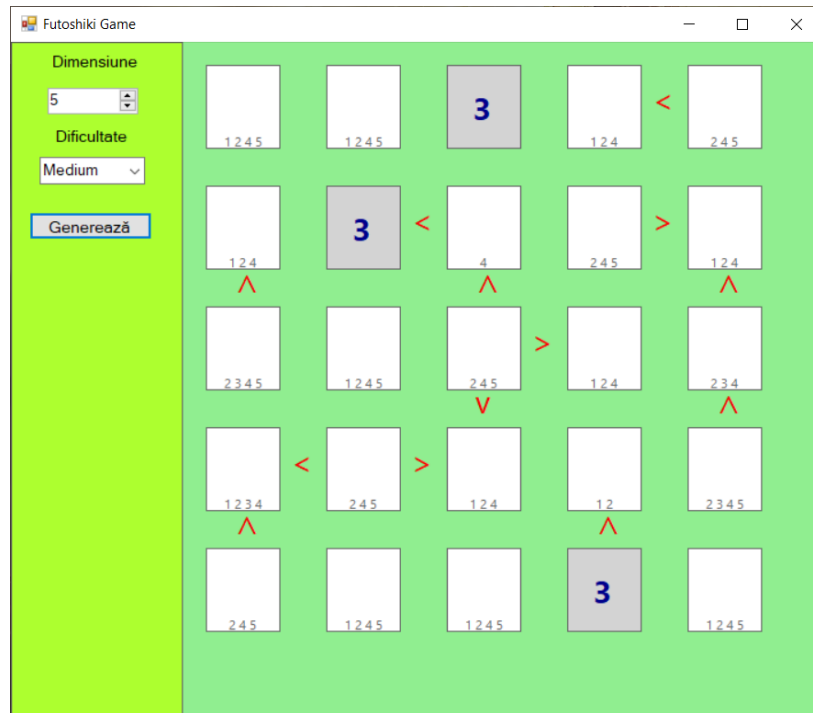
Algoritmul de inițializare și generare a nivelului. Această funcție gestionează crearea unei noi partide prin următorii pași:

1. **Inițializare:** Curăță interfața anterioară și ajustează dimensiunea ferestrei în funcție de mărimea grilei ($N \times N$).
2. **Generarea Soluției:** Creează în memorie o matrice completată corect (un Pătrat Latin), asigurând matematic unicitatea pe linii și coloane.
3. **Construirea Grilei:** Generează controalele vizuale (FutoshikiCell) și le plasează pe ecran. Un procent de 20% dintre acestea sunt dezvăluite ca indicii fixe (IsFixed).
4. **Aplicarea Constrângerilor:** Parcurge grila și adaugă semne de inegalitate ($<$, $>$, $\vee$, $\wedge$) între celule, cu o frecvență determinată de variabila difficulty. Semnele sunt alese prin compararea valorilor din soluția generată anterior, garantând că puzzle-ul nu conține contradicții.
5. **Inițializare CSP:** Apelează RefreshAllDomains pentru a calcula domeniile inițiale prin Forward Checking.

Rezultate

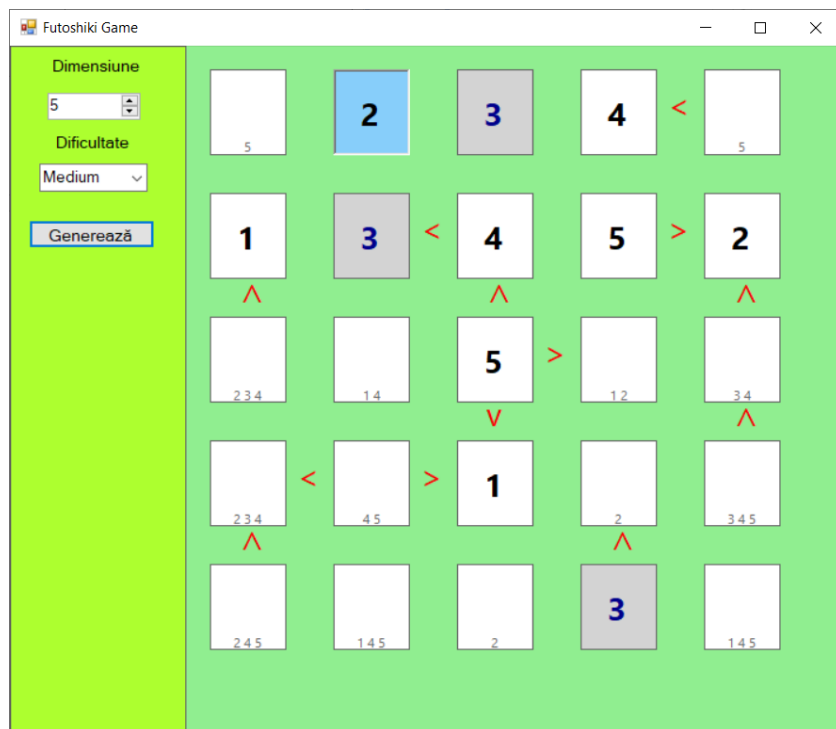
Scenariul 1: Inițializarea Jocului

La generare, algoritmul calculează deja domeniile inițiale. Se observă că sub celulele goale sunt afișate numerele posibile. De exemplu, o celulă care are semnul $<$ față de un "2" fixat, va avea deja eliminate valorile 2, 3, 4, 5 din domeniu.



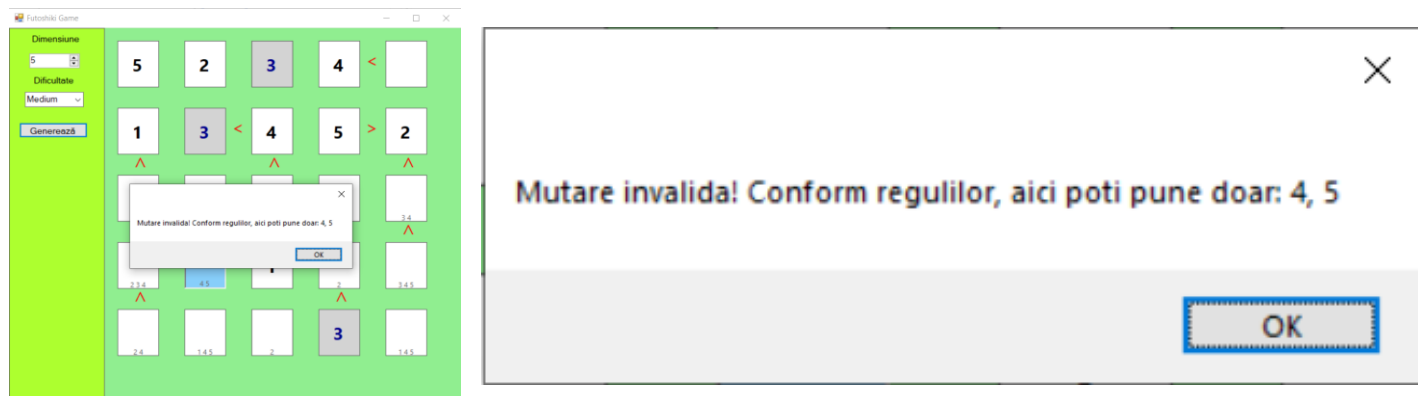
Scenariul 2: Propagarea Constrângerilor (Efectul Forward Checking)

Utilizatorul introduce valoarea 4. Imediat, se observă că cifra 4 dispăre din lista de opțiuni a tuturor celulelor de pe aceeași linie și coloană. Aceasta demonstrează reducerea dinamică a spațiului de căutare.



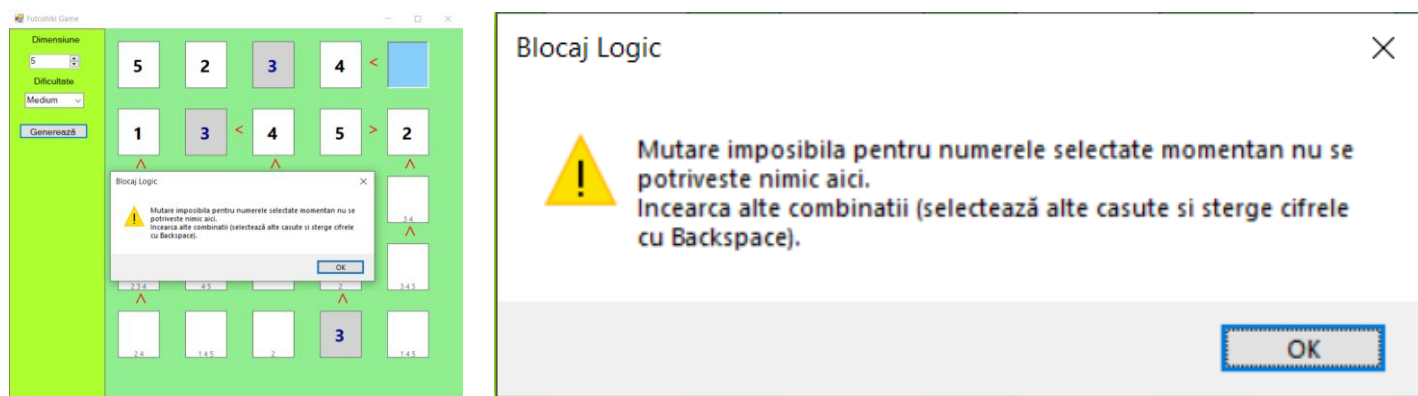
Scenariul 3: Prevenirea Erorilor

Utilizatorul a încercat să introducă o valoare care intră în conflict cu un vecin (ex: $5 < 3$). Algoritmul a respins mutarea, protejând consistența jocului.



Scenariul 4: Detectia Blocajelor (Domeniu Vid)

Deși celula curentă este goală, algoritmul a detectat că, din cauza constrângerilor impuse de vecini, nu mai există nicio valoare legală (domeniul este vid). Utilizatorul este informat că trebuie să revină asupra mutărilor anterioare (backtrack manual).



Concluzii

Proiectul a demonstrat eficiența utilizării algoritmului Forward Checking în jocurile de logică. Față de o implementare simplă, abordarea bazată pe CSP oferă avantaje majore:

1. **Ghidare Inteligentă:** Utilizatorul nu trebuie să ghicească; "pencil marks" (domeniile vizibile) deduse automat simplifică procesul de decizie.
2. **Prevenirea Erorilor:** Sistemul nu permite stări invalide, forțând utilizatorul să respecte regulile la fiecare pas.
3. **Generare Robustă:** Algoritmul de generare inversă asigură că puzzle-urile create sunt întotdeauna corecte și rezolvabile.

Contributii

Rotaru Ionut-Rares

- Clasa CellGrid, Constrangeri
- Implementarea algoritmului din metoda GenerateBoardUI care construiește o tablă de joc validă
- Imbunatatiri vizuale la interfata (culori, centrare)
- Adaugarea logicii de dificultate

Stoian Mario-Daniel

- Proiectarea și implementarea formularului principal
- Implementarea clasei vizuale FutoshikiCell (UserControl), care permite afișarea duală: valoarea curentă (cifră mare) și domeniul de valori posibile (cifre mici/hint-uri).
- Gestionarea evenimentelor de mouse (selecția celulelor) și tastatură
- Implementarea verificărilor în timp real care previn mutările invalide și detectează blocajele
- Implementarea mecanismului RefreshAllDomains care actualizează vizual opțiunile rămase

Github:

<https://github.com/StoianMarioDaniel/Forward-Checking---Futoshiki-Project.git>

Bibliografie

1. Russell, S. J., Norvig, P. (2022). Artificial Intelligence: A Modern Approach, 4th edition, Prentice Hall.
2. Cursul 3 "Probleme de satisfacere a constrângerilor", disciplina Inteligență Artificială.
3. Microsoft Documentation - Windows Forms & .NET Framework.
4. <https://www.geeksforgeeks.org/artificial-intelligence/constraint-satisfaction-problems-csp-in-artificial-intelligence/>